3F7: Information Theory and Coding

Handout 4: Practical source coding: Shannon-Fano coding, Huffman coding, Arithmetic coding

Ramji Venkataramanan

rv285@cam.ac.uk

Michaelmas Term 2025

Designing good codes

In this handout, we will describe some practical compression techniques and analyse their performance.

Recall from the coding theorem: expected code length $L \geq H(X)$.

Thus for a source which can take m values with probabilities $p_1, \ldots, p_m$, we have

$$L = \sum_{i=1}^{m} p_i \ell_i \geq \sum_{i=1}^{m} p_i \log_2 \frac{1}{p_i}$$

- When can this inequality become an equality?
- How do we pick lengths to make it as tight as possible (and therefore L as small as possible)?

Shannon-Fano Coding

Since the codeword lengths ℓ_i are integers, an obvious way to choose them is

$$\ell_i = \left\lceil \log_2 \frac{1}{p_i} \right\rceil,$$

where $\lceil x \rceil$ is the rounding up operation.

Note that $x \leq \lceil x \rceil < x + 1$. Therefore

$$L = \sum_i p_i \ell_i < \sum_i p_i \left(\log_2 \frac{1}{p_i} + 1 \right) = H(X) + 1.$$

Verify that Kraft inequality is satisfied:

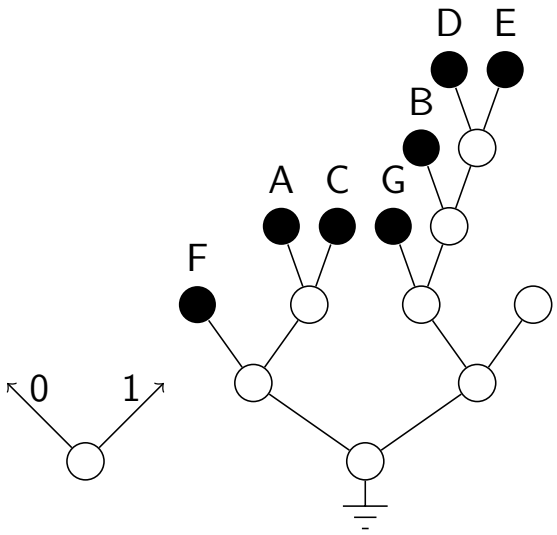
$$\sum_i 2^{-\ell_i} = \sum_i 2^{-\left\lceil \log_2 \frac{1}{p_i} \right\rceil} \leq \sum_i 2^{-\log_2 \frac{1}{p_i}} = \sum_i p_i = 1.$$

Hence we can construct a prefix-free code with this set of lengths.

To construct the code with these code lengths, we simply grow a tree from its root placing codewords on leaves as we reach the required depths.

Example:

x	p_i	$\log(1/p_i)$	ℓ_i
A	0.15	2.73	3
B	0.07	3.83	4
C	0.17	2.55	3
D	0.06	4.05	5
E	0.06	4.05	5
F	0.31	1.68	2
G	0.18	2.47	3



We have shown above that $L < H(X) + 1$, but we can do better in general. How can you improve the code in the above example?

Properties of an optimal prefix-free code

1. The lengths are ordered inversely with the probabilities, i.e.,
if $p_j > p_k$, then $\ell_j \leq \ell_k$.
(If not, we can swap the lengths to get a strictly better code.)

2. The two least probable symbols have the *same length* and are on neighbouring leaves in the binary tree (i.e., they differ only in the last digit).
(Can show by drawing a picture that the expected code length can always be decreased if this is violated.)

Huffman Coding

Huffman is a simple algorithm that gives you an **optimal** prefix-free code for a given set of probabilities.

It uses the above two properties of an optimal code and builds the binary tree *backwards* starting from the leaves.

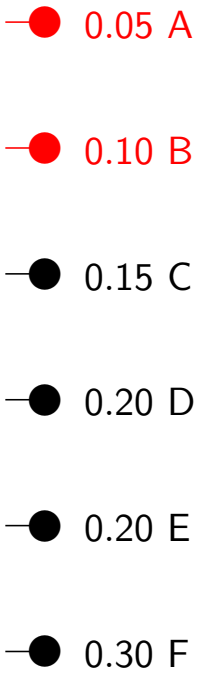
The algorithm is as follows:

1. Take the two least probable symbols in the alphabet. These two symbols will be given the longest codewords, which will have equal length, and differ only in the last digit.

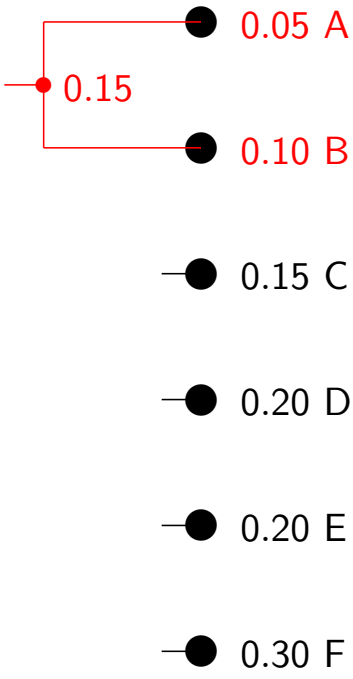
2. Combine these two symbols into a single symbol, and repeat.

We will explain the procedure via an example.

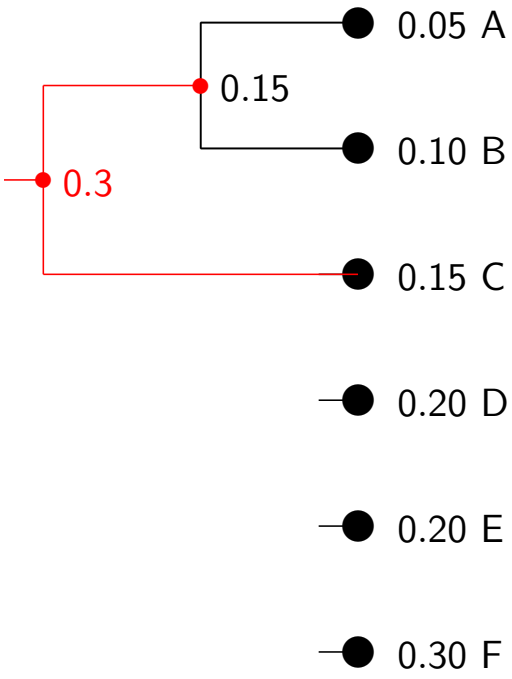
- A source produces symbols from the set $\{A, B, C, D, E, F\}$ with probabilities $\{0.05, 0.1, 0.15, 0.2, 0.2, 0.3\}$.
- List these symbols in increasing order of their probabilities.



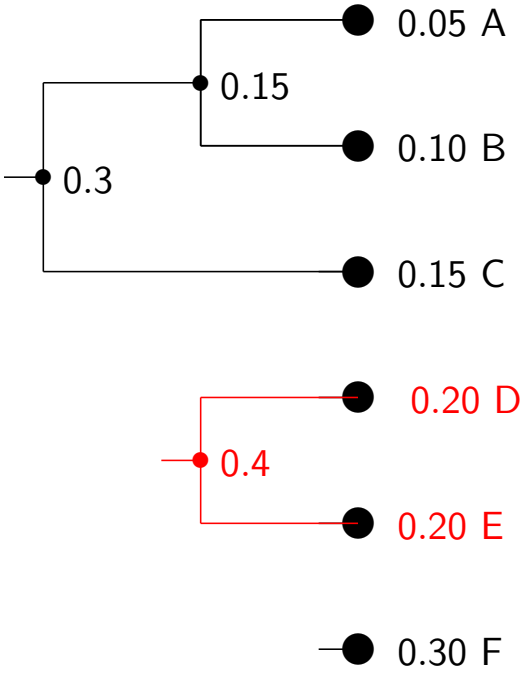
Combine the two symbols with the smallest probabilities, and form a "super-symbol" with the sum of their probabilities.



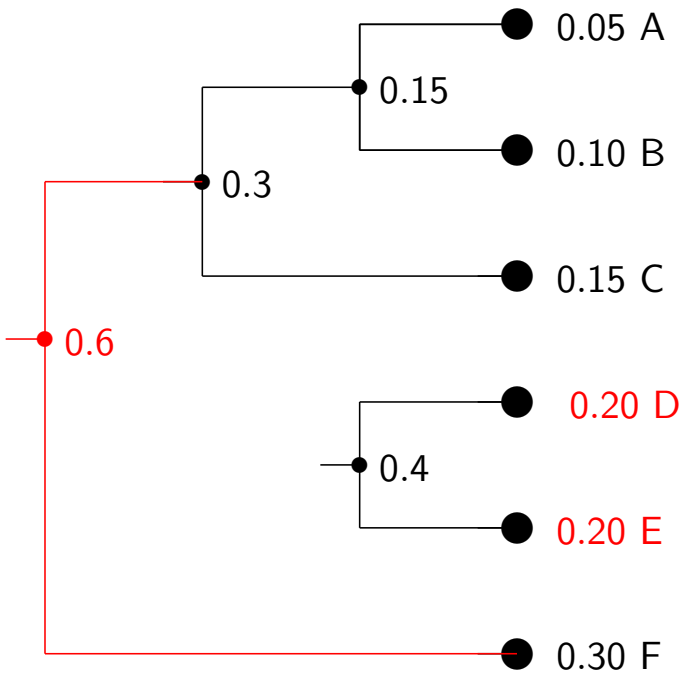
We now have five symbols with probabilities $\{0.15, 0.15, 0.2, 0.2, 0.3\}$. Again, combine the two symbols with the smallest probabilities ...



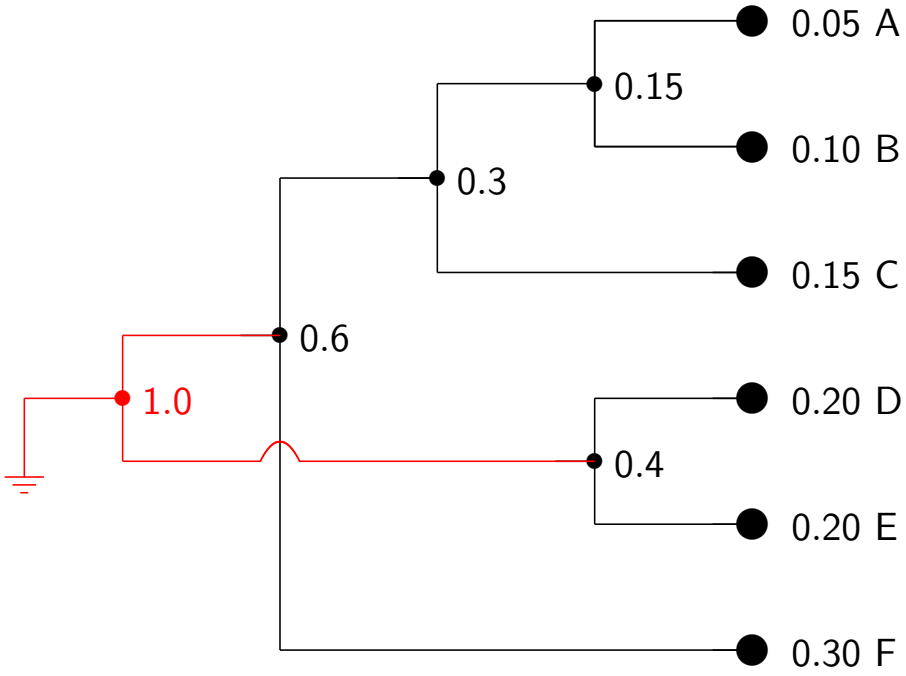
Among the four remaining symbols, D, E have the smallest probabilities $\{0.2, 0.2\}$, so combine them ...



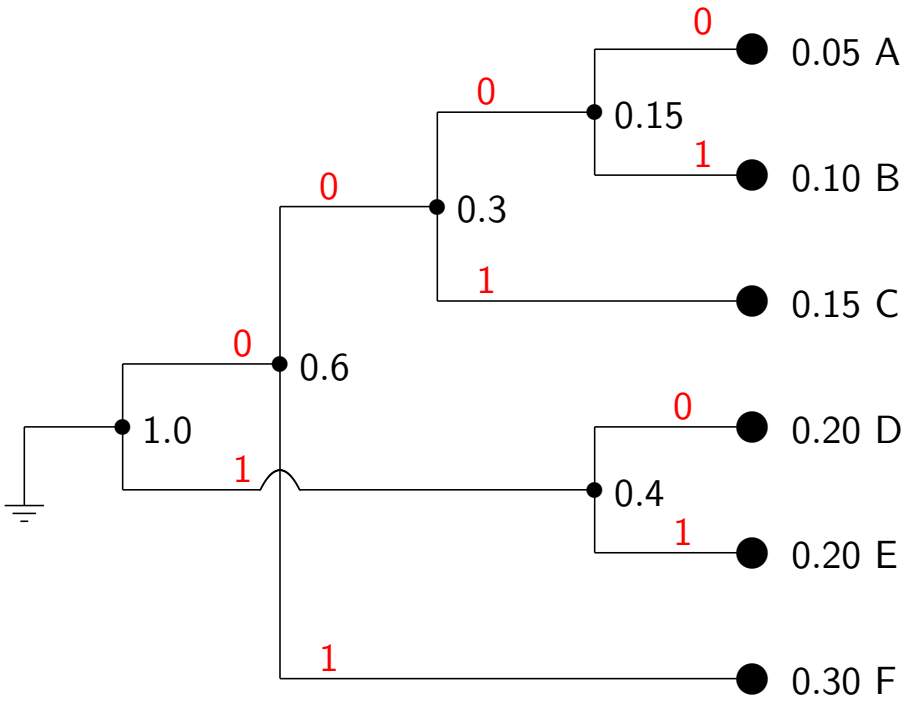
Among the three remaining symbols, the smallest probabilities are $\{0.3, 0.3\}$, so combine them ...



Finally, combine the remaining two symbols . . .



The symbols are the leaves of a tree (at different depths). The final step is to assign the codewords to the symbols using the tree.



The Huffman code is:
$F \to 01$, $E \to 11$, $D \to 10$, $C \to 001$, $B \to 0001$, $A \to 0000$

Verify that the expected codeword length is 2.45 code bits/symbol

Properties of the Huffman Code

- For a source with alphabet of size m, the Huffman algorithm requires $m - 1$ steps of combining the two smallest probabilities at each stage.
- The Huffman code for a given source may not be unique: Besides the fact that we can swap the 1s and 0s in the final code, observe that at any step, ties can be broken arbitrarily. Thus we may have different Huffman codes for the same source, but they will all have the *same* expected code length.

Optimality of Huffman Coding

For a given set of probabilities, there is no prefix-free code that has smaller expected length than the Huffman code.

Proof of Optimality (Not examinable; see Cover & Thomas, Sec. 5.8):
Let the source pmf be $\mathbf{p} = (p_1, \ldots, p_m)$, with $p_1 \geq p_2 \ldots \geq p_m$. Let $\mathbf{p}' = (p_1, \ldots, p_{m-1} + p_m)$ be the pmf formed by combining the last two symbols. Let C_m^* and C_{m-1}^* be *optimal codes* (possibly non-Huffman) for these two pmfs with expected lengths L_m^* and L_{m-1}^*, respectively.
1) Start with C_{m-1}^* and construct a code for the m-symbol source $\mathbf{p}$ by extending the codeword corresponding to $(p_{m-1} + p_m)$ by 0 and 1 to form a new code C_m for $\mathbf{p}$. The expected code length of C_m is

$$L_m = L_{m-1}^* + p_{m-1} + p_m.$$

2) Next, take the optimal code C_m^*, and form a code for $\mathbf{p}'$ by merging the two codewords for the lowest probability symbols $(m-1)$ and m. (From the property of optimal prefix-free codes (p.6), the two codewords being merged are neighbouring leaves.) This new code, called C_{m-1}, has average length

$$L_{m-1} = L_m^* - p_{m-1} - p_m$$

Adding the two equations, we get $(L_{m-1} - L_{m-1}^*) + (L_m - L_m^*) = 0$. Since L_{m-1}^* and L_m^* are optimal, this implies $L_m = L_m^*$ and $L_{m-1} = L_{m-1}^*$.
Thus we have shown that the 'Huffman reduction' in each step is optimal. By induction, starting with the (obvious) optimal code for $m = 2$, this implies that the Huffman procedure yields a code with the optimal expected code length. □

Huffman codes are optimal for coding a *single* random variable X, and have expected code length that is less than $H(X) + 1$. But they have some weaknesses:

- Typically, we want to compress a long source sequence $X_1, X_2, \ldots, X_n$. Even if this source sequence was iid (which is usually not the case), symbol-by-symbol Huffman coding may give you an expected length closer to $H(X) + 1$ bits/symbol rather than $H(X)$.
- To reduce this overhead of up to 1 bit/symbol, we could design a Huffman code for blocks of k symbols. This would give us an overhead of 1 bit/k symbols, or $\frac{1}{k}$ bits/symbol.
- But this comes at the expense of increased complexity. For blocks of k symbols, the alphabet size is $|\mathcal{X}|^k$. Thus the binary tree is much larger. Further, we have decode a block of k symbols at a time, rather than symbol by symbol.

These defects are addressed by Arithmetic coding, a scalable algorithm whose expected code length is very close to the source entropy for large sequences. It can also easily deal with non-iid source distributions such as those used to model text.

Interval coding

Key idea: Each symbol can be represented as an *interval* inside $[0, 1]$, with length of the interval equal to the symbol probability.

A source with m symbols with probabilities $\{p_1, \ldots, p_m\}$ is represented using the m intervals

$$[0, p_1),\ [p_1, p_1 + p_2), \ldots, [\sum_{i=1}^{m-1} p_i, \sum_{i=1}^{m-1} p_i + p_m).$$

Notation: $[0, 1]$ denotes the unit interval with both end-points included. $[0, 1)$ means the interval with the point 0 included, but not 1.

The binary codeword is obtained using the binary representation of the endpoints of each interval.

Example: To represent the interval $[0.17, 0.43)$, convert the end-points to binary. This is $[0.00\overline{101}, 0.01\overline{101})$. Noting that all numbers which in binary are of the form $\mathbf{0.010xyz}\ldots$ lie within the given interval $[0.17, 0.43)$, so we choose $\mathbf{010}$ as the codeword for $[0.17, 0.43)$.

In general, the binary codeword for a symbol with probability p represented by the interval $[a, a + p)$ can be obtained as follows:

1. Find the largest *dyadic* interval of the form $[\frac{j}{2^\ell}, \frac{j+1}{2^\ell})$ that lies within $[a, a + p)$. (Here j, ℓ are integers)
2. Take the binary representation of the lower end-point of the dyadic interval as the codeword. (This will be the integer j converted to binary and represented using ℓ bits.)

Example: Find the binary codewords with interval coding for a random variable which takes values in $\{a, b, c\}$ have probabilities $\{0.2, 0.45, 0.35\}$.

1. $a = [0, 0.2)$ contains dyadic int. $[0.000, 0.001)$ $\Rightarrow$ **000** is the codeword for a.
2. $b = [0.2, 0.65)$ contains dyadic int. $[0.01, 0.10) \Rightarrow$ **01** is the codeword for b.
3. $c = [0.65, 1)$ contains dyadic int. $[0.11, 1.0) \Rightarrow$ **11** is the codeword for c.

Code Length

With ℓ code bits, the dyadic intervals have length $2^{-\ell}$. The dyadic interval has to be contained within an interval of length p. Hence

$$2^{-\ell} \leq p \quad \Rightarrow \quad \lceil \log_2(1/p) \rceil \leq \ell.$$

But sometimes, we'll need $\ell = \left\lceil \log_2 \frac{1}{p} \right\rceil + 1$ bits to get the dyadic interval small enough to fit within the given interval.

Exercise: Find the binary codeword for each of the following intervals of length $p = 0.26$: 1) [0,0.26), and 2) [0.01, 0.27)

In interval coding, the length of the binary codeword for a symbol with probability p is either $\left\lceil \log_2 \frac{1}{p} \right\rceil$ or $\left\lceil \log_2 \frac{1}{p} \right\rceil + 1$ bits.

The expected code length can therefore be bounded as

$$L = \sum_i p_i \ell_i \leq \sum_i p_i (1 + \lceil \log_2(1/p_i) \rceil) < \mathbf{H(X)} + \mathbf{2} \text{ bits/symbol.}$$

Interval Codes are Prefix-free:
Any binary codeword c refers to an interval of the form $[0.c\bar{0}, 0.c\bar{1})$. Note that $0.c\bar{1}$ is the *same* number as adding 1 to the LSB of $0.c$. E.g., 101 refers to the interval $[0.101\bar{0}, 0.101\bar{1}) = [0.101, 0.110)$.

If a binary codeword c was a prefix of another codeword d, then the interval $[0.d\bar{0}, 0.d\bar{1})$ would be a *sub-interval* of $[0.c\bar{0}, 0.c\bar{1})$.

But the intervals used to code each symbol are *disjoint*. Therefore, the codewords of the interval code satisfy the prefix-free property.

Performance:
In terms of expected code-length, the analysis above shows that interval codes are in general not as good as Huffman codes or even Shannon-Fano codes. But interval coding is the basis for arithmetic coding, which is a powerful technique for long source sequences.

Arithmetic Coding

We will explain the procedure via an example. Consider a source producing symbols $X_1, X_2, \ldots, X_n$ which are i.i.d., with each X_i taking values in $\{a, b, c\}$ with probabilities $\{0.2, 0.45, 0.35\}$.

Key Ideas:

- Each length n string $(x_1, \ldots, x_n)$ is represented by a disjoint interval with length equal to the probability of the string.

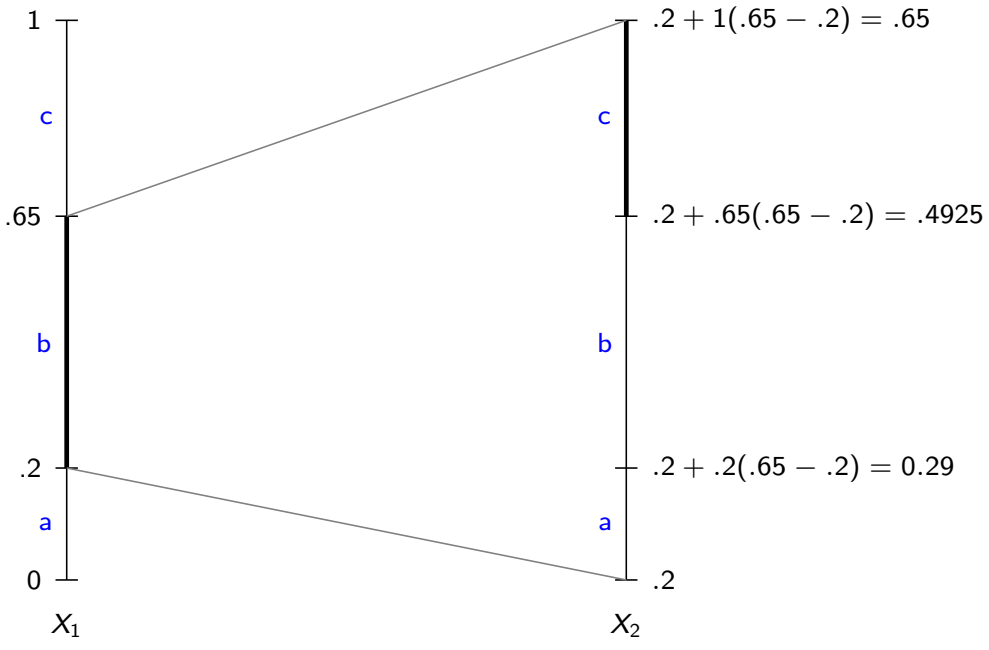
  E.g., $(X_1 = b, X_2 = c)$ corresponds to an interval of length $0.45 \times 0.35 = 0.1575$, and $(X_1 = b, X_2 = a)$ is an interval of length 0.09.

- The interval for $(X_1 = x, X_2 = y)$ is a *sub-interval* of the interval for $X_1 = x$. (Here x, y are any symbols from the alphabet).

  E.g., $(X_1 = b)$ is the interval $[0.2, 0.65)$, and $(X_1 = b, X_2 = c)$ is a subinterval of length 0.1575 *within* $[0.2, 0.65)$.

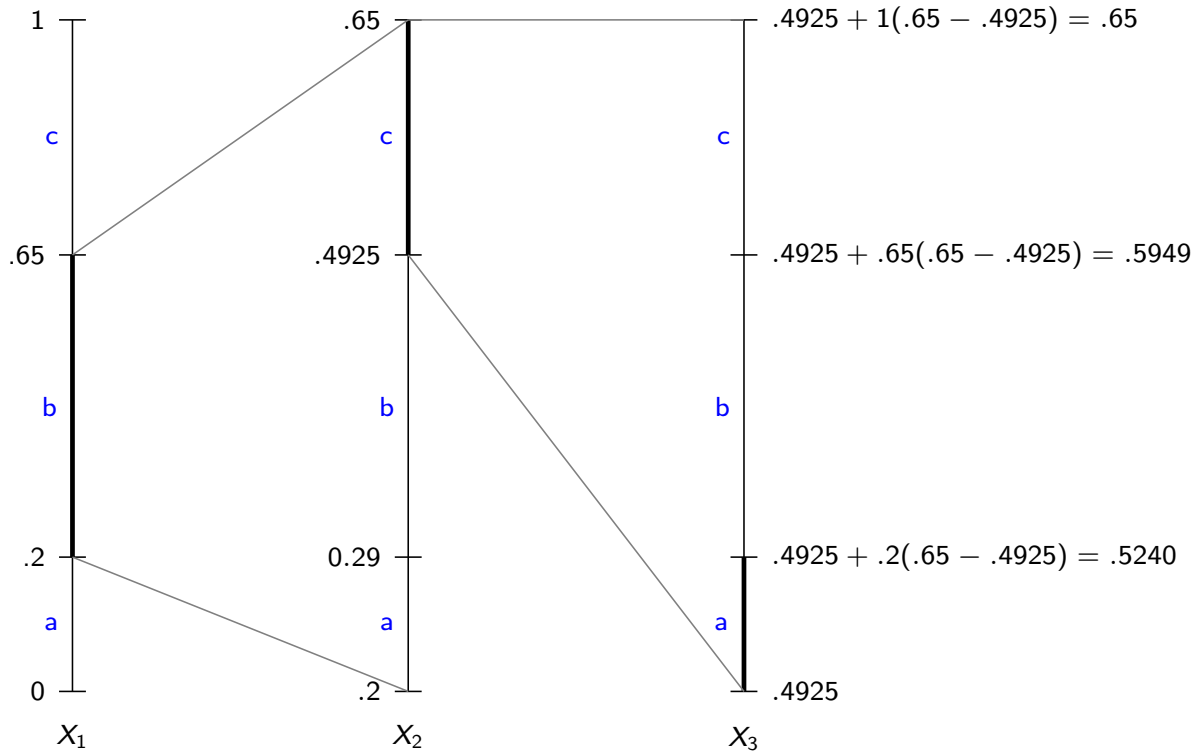
- Similarly, for any symbols x, y, z, $P(X_1 = x, X_2 = y, X_3 = z)$ is a sub-interval of $P(X_1 = x, X_2 = y)$, and so on.

Example: Perform arithmetic coding to find the codeword for the source sequence $(X_1 = b, X_2 = c, X_3 = a, X_4 = c)$.
From interval coding for the first symbol, $X_1 = b$ corresponds to the interval $[0.2, 0.65)$.

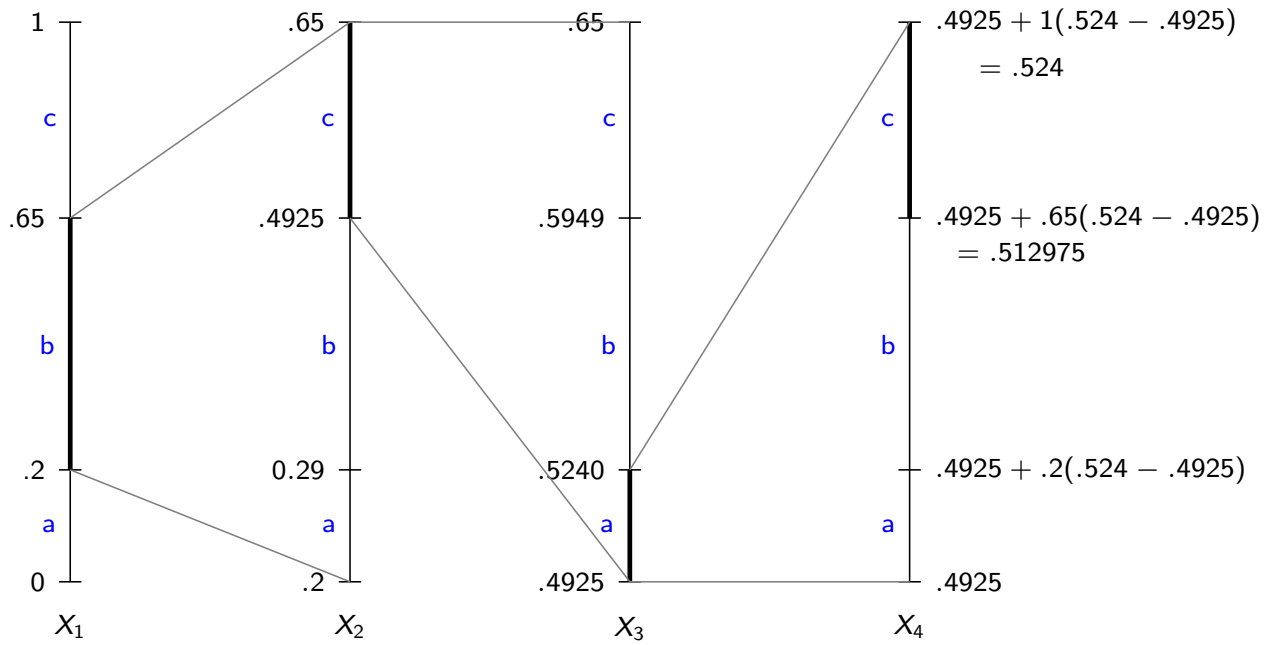


To code X_2, we divide the chosen interval for X_1 in the proportions of the symbol probabilities for X_2 (i.e., $\{0.2, 0.45, 0.3\}$)

To code X_3, we divide the chosen interval for X_2 [0.4925, 0.65) in the proportions of the symbol probabilities for X_3:



To code X_4, we divide the chosen interval for X_3 [0.4925, 0.524) in the proportions of the symbol probabilities for X_4:



Thus the interval representing the sequence (b, c, a, c) is $[0.512975, 0.524)$. (Verify that the width of this interval is equal to the probability of the sequence $= 0.45 \times 0.35 \times 0.2 \times 0.35$.)

Finding the binary codeword:
Recall from the analysis for interval coding:

- The binary codeword for the interval $[0.512975, 0.524)$ is obtained by finding the largest dyadic interval of the form $[\frac{j}{2^\ell}, \frac{j+1}{2^\ell})$ that is contained within the interval.
- The codeword length ℓ is either $\lceil \log_2 \frac{1}{p(x_1,\ldots,x_4)} \rceil$ or $\lceil \log_2 \frac{1}{p(x_1,\ldots,x_4)} \rceil + 1$.
- For the above interval, $\lceil \log_2 \frac{1}{p(x_1,\ldots,x_4)} \rceil = 7$, and the dyadic interval $[\frac{66}{2^7}, \frac{67}{2^7})$ lies within it. Hence the codeword is the binary representation of 66, which is **1000010**

Decoding: Use the binary codeword to sequentially zoom in to the interval, decoding symbols as you go along. For example:
1. $100 \to [0.5, 0.625)$, which implies the first two symbols are b, c.
2. With the next zero, $1000 \to [0.5, 0.5625)$. Proceed in this way, decoding a new symbol whenever possible.
Alternatively, convert the entire binary codeword into a decimal interval, and zoom in to this interval sequentially, decoding as you go along.

Expected Code Length

Arithmetic coding can be performed in this fashion for any sequence of length n, so that any sequence $x_1, \ldots, x_n$ can be represented by an interval of **length** $p(x_1, \ldots, x_n)$, which gives a binary codeword of length at most $\lceil \log_2 \frac{1}{p(x_1,\ldots,x_n)} \rceil + 1$.

Therefore the expected code length for length n sequences is bounded as

$$L_n = \sum_{x^n} p(x^n)\ell(x^n) < \sum_{x^n} p(x^n) \left(\log_2 \frac{1}{p(x_1, \ldots, x_n)} + 2 \right) = H(X^n) + 2$$

Therefore the expected code length per symbol is

$$\frac{L_n}{n} < \frac{H(X^n)}{n} + \frac{2}{n} = H(X) + \frac{2}{n},$$

where the last equality holds if $X_1, X_2, \ldots$ are iid $\sim P_X$

With growing n, arithmetic coding can achieve expected code length/symbol that is arbitrarily close to the source entropy!

We have seen three ways of compressing an iid source X with close to optimal expected codelength:

1. Using separate binary codebooks for typical and non-typical sequences (Handout 2). For any $\epsilon > 0$, can get expected codelength within $H(X) + \epsilon$ bits/source symbol with sequence length n large enough.

2. Using a Shannon-Fano/Huffman code over blocks of k symbols. Expected code length $H(X) + \frac{1}{k}$ bits/source symbol, which can be made close to $H(X)$ by taking k to be large

3. Arithmetic coding: to compress source sequence of length n, expected codelength is within $\frac{2}{n}$ of $H(X)$

How do these compare in implementation complexity ?

Arithmetic coding for non-iid sources

Consider a source that produces symbols in alphabet $\{a_1, \ldots, a_m\}$ with a known distribution

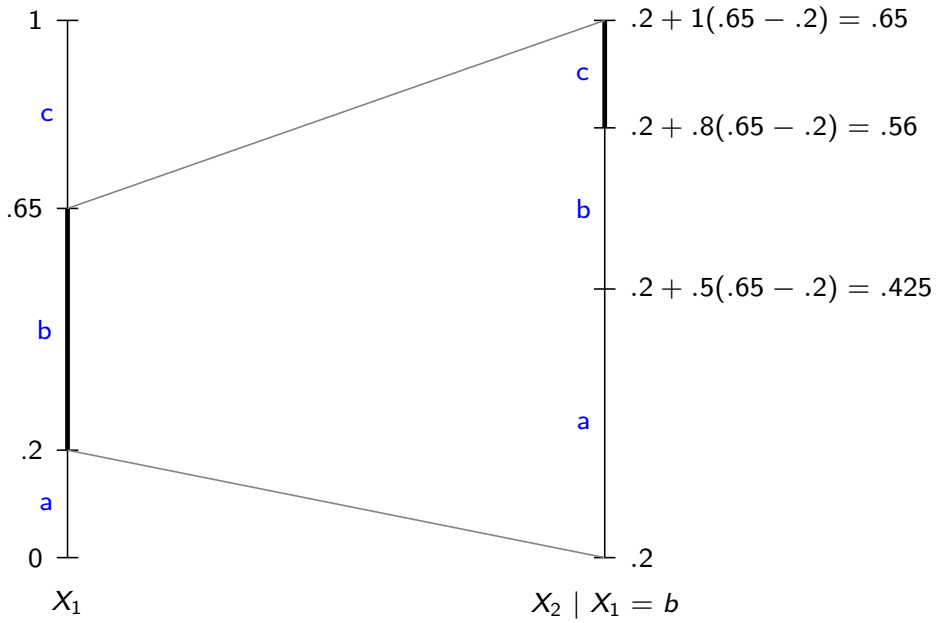
$$P(x_1)P(x_2 \mid x_1) \ldots P(x_n \mid x_1, \ldots, x_{n-1})$$

Such models are frequently used for text where the distribution of a symbol x_n depends on the most recent *context* $x_1, \ldots, x_{n-1}$.

The arithmetic coding algorithm can be easily extended to such sources. As before, consider the source with alphabet $\{a, b, c\}$ with $P(X_1 = a) = 0.2, P(X_1 = b) = 0.45, P(X_1 = c) = 0.35$. Suppose that three conditional distributions $P(X_2|X_1 = a), P(X_2|X_1 = b), P(X_2|X_1 = c)$ are different, and say

$$P(X_2 = a|X_1 = b) = 0.5,\ P(X_2 = b|X_1 = b) = 0.3,$$
$$P(X_2 = c|X_1 = b) = 0.2$$

We find the interval for the source sequence (b, c) as follows.



To code X_2, we divide the chosen interval for X_1 in the proportions of the symbol probabilities for $X_2 \mid X_1$ (i.e., $\{0.5, 0.3, 0.2\}$)

In general, to code symbol X_n, use the *conditional* distribution of X_n given the observed $(x_1, \ldots, x_{n-1})$

The Arithmetic Coding Algorithm

- Let the source alphabet $\mathcal{A}$ be $\{a_1, \ldots, a_m\}$.
- We are given a source sequence $x_1, x_2, \ldots, x_n$ where each $x_i \in \mathcal{A}$.
- Both encoder and decoder know the conditional distributions P_{X_1}, $P_{X_2|X_1}, \ldots, P_{X_n|X_1,\ldots,X_{n-1}}$.
- The encoding algorithm computes the interval using the following lower and upper cumulative probabilities.

For $i = 1, \ldots, m$ and for $k = 1, \ldots, n$, we define

$$L_k(a_i \mid x_1, \ldots, x_{k-1}) := \sum_{i'=1}^{i-1} P(X_k = a_{i'} \mid X_1 = x_1, \ldots, X_{k-1} = x_{k-1})$$

$$U_k(a_i \mid x_1, \ldots, x_{k-1}) := \sum_{i'=1}^{i} P(X_k = a_{i'} \mid X_1 = x_1, \ldots, X_{k-1} = x_{k-1})$$

The encoding algorithm to find the interval for the given sequence $x_1, x_2, \ldots, x_n$ is as follows.

$lo := 0.0$
$hi := 1.0$
$p := hi - lo$
`for` $k = 1$ `to` n $\{$
  Compute $L_k(x_k \mid x_1, \ldots, x_{k-1})$, $U_k(x_k \mid x_1, \ldots, x_{k-1})$
  $hi := lo + p\, U_k(x_k \mid x_1, \ldots, x_{k-1})$
  $lo := lo + p\, L_k(x_k \mid x_1, \ldots, x_{k-1})$
  $p := hi - lo$
$\}$

Note that this is just a generalization of the example we used to illustrate the algorithm.

Finite precision issues

The arithmetic encoding algorithm above assumes an infinite precision computer:

- As n grows, the length of the interval corresponding to a sequence $(x_1, \ldots, x_n)$ shrinks
- Hence the number of digits needed to accurately store the values *lo* and *hi* grows with n
- With everyday computers, precision issues become relevant even for moderately large n

With some simple but clever fixes, the algorithm can be adapted to work with a finite precision computer: see Section 6 of lab handout

Arithmetic coding summary

- Can achieve compression rates very close to the source entropy, with complexity scaling *linearly* with sequence length
- It does require you to know (or at least estimate) the conditional distributions of the source. But this approach fits well with machine learning techniques that can mine huge quantities of data to build good probabilistic models, e.g., Large Language Models. A recent example: `https://arxiv.org/abs/2306.04050`
- Note that the assumed distribution of the source doesn't need to be the true one, it only needs to be the same at both encoder and decoder
- Arithmetic coding works even when you generate the source conditional distributions on the fly, based on what has been so observed so far. Given the generating rule, the decoder will also generate the required conditional distributions as it reconstructs the sequence

References for arithmetic coding:


D. J. C. MacKay,
Information theory, inference, and learning algorithms, Chap. 6


I. H. Witten, R. M. Neal, J. G. Cleary
Arithmetic Coding for Data Compression, Communications of the ACM, June 1987


A. Moffat, R. M. Neal, I. H. Witten
Arithmetic Coding Revisited, ACM Transactions on Information Systems, July 1998

Chapter 6 of MacKay's book also discusses Lempel-Ziv coding, which is yet another efficient data compression technique.

(You can now do all the questions in Examples Paper 1.)